

MEDIATEK

MT8676 Android FBE Debug SOP

版本: 1.0
出版日期: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2025-07-11	晏晓阳	正式版

Table of Contents

版本记录 2

Table of Contents 3

1 与 FBE 相关的开机流程 4

 1.1 Init 流程 4

 1.1.1 early-fs 5

 1.1.2 late-fs 5

 1.1.3 post-fs-data 10

2 检查 FBE 相关的设定 12

 2.1 检查配置文件 12

 2.1.1 ProjectConfig.mk 12

 2.1.2 fstab 12

 2.4 验证 FBE 是否有开启 12

 2.4.1 查看 property 12

 2.4.2 查看 CE 文件是否为密文 13

3 FBE 相关的问题 14

 3.1 如何确认是做加密还是解密 14

 3.2 如何关闭 FBE 功能 14

 3.3 USB 全量升级，恢复出厂设置，第二次开机启动后 FBE Key 异常 14

 3.4 执行 metadata 加密时候，fs_mgr 卸载 data 分区失败 15

 3.5 执行 metadata 加密时候，异常断电，data 分区加密未完成 15

 3.6 执行 metadata 加密过程中，metadata key 被破坏 15

 3.7 后续开机解密流程，如果异常如何恢复，怎样确认是加密问题还是解密问题，或是分区被破坏 16

4 其他网上资料 17

Exhibit 1 Terms and Conditions 18

1 与 FBE 相关的开机流程

Android FBE 是 Android 系统的原生功能。FBE 的加解密过程在设备开机的 init 阶段进行。因此，要了解 FBE 相关的流程，首先需要熟悉开机时 init 的流程。

1.1 Init 流程

根据开机流程，init 阶段会解析 `init.rc` 中的 action。图 1-1 是开机 init 进程的流程图。

Init

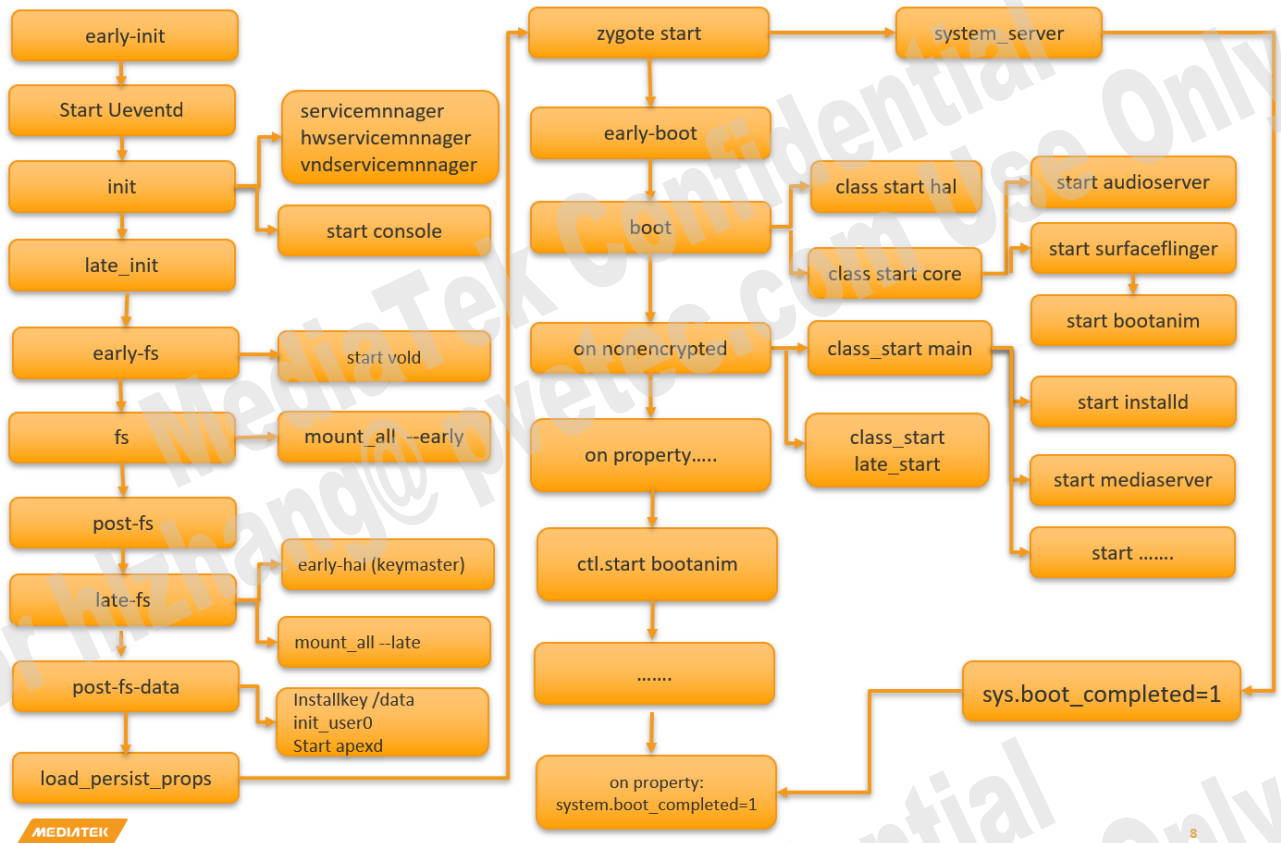


图 1-1. 开机 init 流程图

对于 FBE，需要关注的开机阶段主要有如下几个：

1. early-fs
 - start vold
2. late-fs
 - early-hal (keymaster)
 - mount_all --late

3. post-fs-data
 - installkey /data
 - init_user0

1.1.1 early-fs

vold 会在 on early-fs 阶段启动，后续 FBE 加解密的动作都将由 vold 发起。

1.1.2 late-fs

1. 在 on late-fs 阶段，主要关注两个动作：

- early-hal (keymaster)：这个动作会启动 keymaster，为后续加解密做准备。
- mount -late：这个动作主要用于挂载 userdata 分区，FBE 加解密正是针对 userdata 分区进行的。

2. mount userdata：首先尝试挂载 userdata 分区，并调用 prepare_fs_for_mount 进行准备。

platform/superproject/main > main > system/core/fs_mgr/fs_mgr.cpp

Files	Outline	fs_mgr.cpp
fs_mgr		926 // attempted_idx: On return, will indicate which fstab entry
include		927 // succeeded. In case of failure, it will be the start_idx.
libdm		928 // Sets errno to match the 1st mount failure on failure.
libfiemap		929 static bool mount_with_alternatives(Fstab& fstab, int start_idx, bool interrupted, int* end_idx,
libfs_avb		930 int* attempted_idx) {
libfstab		931 unsigned long i;
liblbp		932 int mount_errno = 0;
libsnapshot		933 bool mounted = false;
libstorage_literals		934
libvbmata		935 // Hunt down an fstab entry for the same mount point that might succeed.
tests		936 for (i = start_idx;
tools		937 // We required that fstab entries for the same mountpoint be consecutive.
.clang-format		938 i < fstab.size() && fstab[start_idx].mount_point == fstab[i].mount_point; i++) {
Android.bp		939 // Don't try to mount/encrypt the same mount point again.
NOTICE		940 // Deal with alternate entries for the same point which are required to be all following
OWNERS		941 // each other.
README.overlays.md		942 if (mounted) {
TEST_MAPPING		943 LINFO << __FUNCTION__ << "(): skipping fstab dup mountpoint=" << fstab[i].mount_point
blockdev.cpp		944 << " rec[" << i << ".fs_type=" << fstab[i].fs_type << " already mounted as "
blockdev.h		945 << fstab[*attempted_idx].fs_type;
clean_scratch_files.rc		946 continue;
file_wait.cpp		947 }
fs_mgr.cpp		948 }
fs_mgr_dm_linear.cpp		949 if (interrupted) {
fs_mgr_format.cpp		950 LINFO << __FUNCTION__ << "(): skipping fstab mountpoint=" << fstab[i].mount_point
fs_mgr_overlayfs_control.cpp		951 << " rec[" << i << ".fs_type=" << fstab[i].fs_type
fs_mgr_overlayfs_control.h		952 << " (previously interrupted during encryption step)";
fs_mgr_overlayfs_mount.cpp		953 continue;
		954 }
		955 // fstab[start_idx].blk_device is already updated to /dev/dm-<N> by
		956 // AVB related functions. Copy it from start_idx to the current index i.
		957 if ((i != start_idx) && fstab[i].fs_mgr_flags.logical &&
		958 fstab[start_idx].fs_mgr_flags.logical &&
		959 (fstab[i].logical_partition_name == fstab[start_idx].logical_partition_name)) {
		960 fstab[i].blk_device = fstab[start_idx].blk_device;
		961 }
		962 }
		963 int fs_stat = prepare_fs_for_mount(fstab[i].blk_device, fstab[i]);
		964 if (fs_stat & FS_STAT_INVALID_MAGIC) {
		965 ERROR << __FUNCTION__
		966 << "(): skipping mount due to invalid magic, mountpoint=" << fstab[i].mount_point
		967 << " blk_dev=" << realpath(fstab[i].blk_device) << " rec[" << i
		968 << ".fs_type=" << fstab[i].fs_type;
		969 }

3. 上面 mount userdata 的结果有两种情况：

- 首次开机（刷机或执行 factory reset 之后）：在这种情况下，挂载操作返回成功，表示分区没有加密，首次尝试挂载时就成功了（mret=true）。此时需要判断是否需要加密，因为在首次开机时分区未加密，挂载成功后需要先卸载分区，然后进入加密流程。
- 非首次开机：经过首次开机后，data 分区处于加密的状态。此时检查文件系统会返回错误，出现 invalid magic 的错误。这是正常现象，并不是文件系统损坏的标志。

```

1527
1528     int last_idx_inspected;
1529     int top_idx = 1;
1530     int attempted_idx = -1;
1531
1532     bool mret = mount_with_alternatives(*fstab, i, &last_idx_inspected, &attempted_idx);
1533     auto& attempted_entry = (*fstab)[attempted_idx];
1534     i = last_idx_inspected;
1535     int mount_errno = errno;
1536
1537     // Handle success and deal with encryptability.
1538     if (mret) {
1539         int status = handle_encryptable(attempted_entry);
1540
1541         if (status == FS_MGR_MNTALL_FAIL) {
1542             // Fatal error - no point continuing.
1543             return {status, userdata_mounted};
1544         }
1545
1546         if (status != FS_MGR_MNTALL_DEV_NOT_ENCRYPTABLE) {
1547             if (encryptable != FS_MGR_MNTALL_DEV_NOT_ENCRYPTABLE) {
1548                 // Log and continue
1549                 ERROR << "Only one encryptable/encrypted partition supported";
1550             }
1551             encryptable = status;
1552             if (status == FS_MGR_MNTALL_DEV_NEEDS_METADATA_ENCRYPTION) {
1553                 fs_mgr_set_blk_ro(attempted_entry.blk_device, false);
1554                 if (!call_vdc(["cryptfs", "encryptFstab", attempted_entry.blk_device,
1555                             attempted_entry.mount_point, wiped ? "true" : "false",
1556                             attempted_entry.fs_type, attempted_entry.zoned_device],
1557                             nullptr)) {
1558                     ERROR << "Encryption failed";
1559                     set_type_property(encryptable);
1560                     return {FS_MGR_MNTALL_FAIL, userdata_mounted};
1561                 }
1562             }
1563         }
1564     }
  
```

卸载分区后，会进入 vdc encryptFstab 流程。需要注意的是，无论是对分区进行加密还是解密，最终调用的 API 都是相同的。

“cryptfs” encryptFstab ：对分区做加密

“cryptfs” mountFstab ：对分区做解密

platform/superproject/main > main > system/core/fs_mgr/fs_mgr.cpp

Files	Outline	fs_mgr.cpp
fs_mgr		1596
include		1597
libdm		1598
libfiemap		1599
libfs_avb		1600
libfstab		1601
liblbp		1602
libsnapshot		1603
libstorage_literals		1604
libvbmata		1605
tests		1606
tools		1607
.clang-format		1608
Android.bp		1609
		1610
		1611
		1612
		1613
		1614
		1615
		1616
		1617
		1618

```

1596
1597     if (status != FS_MGR_MNTALL_DEV_NOT_ENCRYPTABLE) {
1598         if (encryptable != FS_MGR_MNTALL_DEV_NOT_ENCRYPTABLE) {
1599             // Log and continue
1600             LERROR << "Only one encryptable/encrypted partition supported";
1601         }
1602         encryptable = status;
1603         if (status == FS_MGR_MNTALL_DEV_NEEDS_METADATA_ENCRYPTION) {
1604             fs_mgr_set_blk_ro(attempted_entry.blk_device, false);
1605             if (!call_vdc("cryptfs", "encryptFstab", attempted_entry.blk_device,
1606                 attempted_entry.mount_point, wiped ? "true" : "false",
1607                 attempted_entry.fs_type,
1608                 attempted_entry.fs_mgr_flags.is_zoned ? "true" : "false",
1609                 std::to_string(attempted_entry.length),
1610                 android::base::Join(attempted_entry.user_devices, ' '),
1611                 android::base::Join(attempted_entry.device_aliases, ' ')),
1612                 nullptr)) {
1613                 LERROR << "Encryption failed";
1614                 set_type_property(encryptable);
1615                 return FS_MGR_MNTALL_FAIL;
1616             }
1617         }
1618     }

```

上述流程中，对 userdata 进行加密和解密最终都会进入以下流程：

platform/superproject > android14-qpr3-release > system/vold/vdc.cpp

Files	Outline	vdc.cpp
vold		122
MetadataCrypt.h		123
MoveStorage.cpp		124
MoveStorage.h		125
NetlinkHandler.cpp		126
NetlinkHandler.h		127
NetlinkManager.cpp		128
NetlinkManager.h		129
OWNERS		130
PREUPLOAD.cfg		131
Process.cpp		132
		133
		134
		135
		136
		137
		138

```

122     checkStatus(args, vold->shutdown());
123 } else if (args[0] == "volume" && args[1] == "reset") {
124     checkStatus(args, vold->reset());
125 } else if (args[0] == "volume" && args[1] == "getStorageSize") {
126     int64_t size;
127     checkStatus(args, vold->getStorageSize(&size));
128     LOG(INFO) << size;
129 } else if (args[0] == "cryptfs" && args[1] == "bindkeys") {
130     bindkeys(args, vold);
131 } else if (args[0] == "cryptfs" && args[1] == "mountFstab" && args.size() == 5) {
132     checkStatus(args, vold->mountFstab(args[2], args[3], args[4]));
133 } else if (args[0] == "cryptfs" && args[1] == "encryptFstab" && args.size() == 7) {
134     auto shouldFormat = android::base::ParseBool(args[4]);
135     if (shouldFormat == android::base::ParseBoolResult::kError) exit(EINVAL);
136     checkStatus(args, vold->encryptFstab(args[2], args[3],
137         shouldFormat == android::base::ParseBoolResult::kTrue,
138         args[5], args[6]));

```

从上述流程进入以下步骤时，如前所述，加密和解密最终都会调用 `fsencrypt_mount_metadata_encrypted` 函数。

区别在于第三个参数，该参数用于指示是否需要加密。

platform/superproject/main > main > system/vold/VoldNativeService.cpp

Files	Outline	VoldNativeService.cpp
vold		629
NetlinkHandler.h		630 binder::Status VoldNativeService::mountFstab(const std::string& blkDevice,
NetlinkManager.cpp		631 const std::string& mountPoint, bool isZoned,
NetlinkManager.h		632 const std::vector<std::string>& userDevices) {
OWNERS		633 ENFORCE_SYSTEM_OR_ROOT;
PREUPLOAD.cfg		634 ACQUIRE_LOCK;
Process.cpp		635
Process.h		636 return translateBool(fscrypt_mount_metadata_encrypted(blkDevice, mountPoint, false, false,
TEST_MAPPING		637 "null", isZoned, userDevices, {}, 0));
Utils.cpp		638 }
Utils.h		639
VendorVoldNativeService.cpp		640 binder::Status VoldNativeService::encryptFstab(const std::string& blkDevice,
VendorVoldNativeService.h		641 const std::string& mountPoint, bool shouldFormat,
VoldNativeService.cpp		642 const std::string& fsType, bool isZoned,
VoldNativeService.h		643 const std::vector<std::string>& userDevices,
VoldNativeServiceValidation.cpp		644 const std::vector<bool>& deviceAliased,
		645 int64_t length) {
		646 ENFORCE_SYSTEM_OR_ROOT;
		647 ACQUIRE_LOCK;
		648
		649 return translateBool(fscrypt_mount_metadata_encrypted(blkDevice, mountPoint, true, shouldFormat,
		650 fsType, isZoned, userDevices,
		651 deviceAliased, length));
		652 }
		653 }

4. 从上述位置进入 vold 的 system/vold/MetadataCrypt.cpp 中的 fscrypt_mount_metadata_encrypted 函数。在这个流程中，主要有三个步骤：

- Step 1 和 Step 2：根据是否需要加密来决定设备是加密 key 还是解密 key。
- Step 3：创建用于加解密的设备。

platform/superproject/main > main > system/vold/MetadataCrypt.cpp

Files	Outline	MetadataCrypt.cpp
vold		343 if (!user_devices.empty()) {
bench		344 default_metadata_key_dir = default_metadata_key_dir + "/default";
binder		345 }
fs		346 1. auto gen = needs_encrypt ? makeGen(options) : neverGen();
model		347 KeyBuffer key;
tests		348 2. if (!read_key(default_metadata_key_dir, gen, true, &key)) {
.clang-format		349 LOG(ERROR) << "read_key failed in mountFstab";
Android.bp		350 return false;
AppFuseUtil.cpp		351 }
AppFuseUtil.h		352
Benchmark.cpp		353 std::string crypto_blkdev;
BenchmarkGen.h		354 uint64_t nr_sec;
Checkpoint.cpp		355 3. if (!create_crypto_blk_dev(kDmNameUserdata, blk_device, key, options, &crypto_blkdev, &nr_sec,
Checkpoint.h		356 true)) {
CleanSpec.mk		357 LOG(ERROR) << "create_crypto_blk_dev failed in mountFstab";
CryptoType.cpp		358 return false;
CryptoType.h		359 }
EncryptInplace.cpp		360
EncryptInplace.h		361 // create dm-default-key for user devices
FileDeviceUtils.cpp		362 std::vector<std::string> crypto_user_blkdev;
FileDeviceUtils.h		363 for (auto& device : user_devices) {
FsCrypt.cpp		364 std::string name = Basename(device);
		365 auto metadata_key_dir = data_rec->metadata_key_dir + "/" + name;
		366
		367 if (!read_key(metadata_key_dir, gen, false, &key)) {
		368 LOG(ERROR) << "read_key failed with zoned device: " << device;
		369 return false;
		370 }
		371 std::string crypto_blkdev_arg;
		372 if (!create_crypto_blk_dev(name, device, key, options, &crypto_blkdev_arg, &nr_sec, true)) {
		373 LOG(ERROR) << "fscrypt_mount_metadata_encrypted: failed with device: " << device;
		374 return false;
		375 }
		376 crypto_user_blkdev.push_back(crypto_blkdev_arg.c_str());
		377 }

创建完加解密设备后，调用 mount_via_fs_mgr 并带入 crypto_blkdev。

```
platform/superproject/main > > main > system/vold/MetadataCrypt.cpp

Files Outline MetadataCrypt.cpp

vold
└─ bench
└─ binder
└─ fs
└─ model
└─ tests
.clang-format
Android.bp

410     }
411     if (!marker.Remove()) return false;
412 }
413
414 LOG(DEBUG) << "Mounting metadata-encrypted filesystem:" << mount_point;
415 mount_via_fs_mgr(mount_point.c_str(), crypto_blkdev.c_str(), needs_encrypt);
416
417 // Record that there's at least one fstab entry with metadata encryption
418 if (!android::base::SetProperty("ro.crypto.metadata.enabled", "true")) {
419     LOG(WARNING) << "failed to set ro.crypto.metadata.enabled"; // This isn't fatal
420 }
421 return true;
422 }
```

这个流程会进入 FS 的挂载过程。

```
platform/superproject/main > > main > system/vold/MetadataCrypt.cpp

Files Outline MetadataCrypt.cpp

vold
└─ bench
└─ binder
└─ fs
└─ model
└─ tests
.clang-format
Android.bp
AppFuseUtil.cpp
AppFuseUtil.h
Benchmark.cpp
Benchmark.h
BenchmarkGen.h

99 static bool mount_via_fs_mgr(const char* mount_point, const char* blk_device, bool needs_encrypt) {
100     // fs_mgr_do_mount runs fsck. Use setexeccon to run trusted
101     // partitions in the fsck domain.
102     if (setexeccon(android::vold::sFsckContext)) {
103         PLOG(ERROR) << "Failed to setexeccon";
104         return false;
105     }
106     auto mount_rc = fs_mgr_do_mount(&fstab_default, mount_point, blk_device,
107                                     android::vold::cp_needsCheckpoint(), needs_encrypt);
108     if (setexeccon(nullptr)) {
109         PLOG(ERROR) << "Failed to clear setexeccon";
110         return false;
111     }
112     if (mount_rc != 0) {
113         LOG(ERROR) << "fs_mgr_do_mount failed with rc " << mount_rc;
114         return false;
115     }
116     LOG(DEBUG) << "Mounted " << mount_point;
117     return true;
118 }
```

- 最后，流程回到 fs_mgr 中的 fs_mgr_do_mount。此时，如果 blk_device 需要解密，系统会先进行解密，确保读取到的 magic 是正确的，然后进入第二步的 __mount。

platform/superproject/main > main > system/core/fs_mgr/fs_mgr.cpp

Files Outline <| fs_mgr.cpp

```

1847
1848 // Now mount it where requested */
1849 mount_point = fstab_entry.mount_point;
1850
1851 1. int fs_stat = prepare_fs_for_mount(n_blk_device, fstab_entry, mount_point);
1852
1853 if (fstab_entry.fs_mgr_flags.avb) {
1854     if (!avb_handle) {
1855         avb_handle = AvbHandle::Open();
1856         if (!avb_handle) {
1857             LERROR << "Failed to open AvbHandle";
1858             return FS_MGR_DOMNT_FAILED;
1859         }
1860     }
1861     if (avb_handle->SetUpAvbHashtree(&fstab_entry, true /* wait_for_verity_dev */) ==
1862         AvbHashtreeResult::kFail) {
1863         LERROR << "Failed to set up AVB on partition: " << fstab_entry.mount_point
1864             << ", skipping!";
1865         // Skips mounting the device.
1866         continue;
1867     }
1868 } else if (!fstab_entry.avb_keys.empty()) {
1869     if (AvbHandle::SetUpStandaloneAvbHashtree(&fstab_entry) == AvbHashtreeResult::kFail) {
1870         LERROR << "Failed to set up AVB on standalone partition: "
1871             << fstab_entry.mount_point << ", skipping!";
1872         // Skips mounting the device.
1873         continue;
1874     }
1875 }
1876
1877 int retry_count = 2;
1878 while (retry_count-- > 0) {
1879     2. if (!__mount(n_blk_device, mount_point, fstab_entry)) {
1880         fs_stat &= FS_STAT_FULL_MOUNT_FAILED;
1881         log_fs_stat(fstab_entry.blk_device, fs_stat);
1882         return FS_MGR_DOMNT_SUCCESS;
1883     } else {
1884         if (retry_count <= 0) break; // run check_fs only once
1885         if (!first_mount_errno) first_mount_errno = errno;
1886         mount_errors++;
1887         PERROR << "Cannot mount filesystem on " << n_blk_device << " at " << mount_point
1888             << " with fstype " << fstab_entry.fs_type;
1889         fs_stat |= FS_STAT_FULL_MOUNT_FAILED;

```

1.1.3 post-fs-data

在 data 分区挂载成功后，系统会进入 post-fs-data 阶段。在 init.rc 中，会执行 installkey /data 的操作，这个操作会调用 do_installkey 函数，并进入 VDC enablefilecrypto 的流程。

platform/superproject/main > main > system/core/init/builtins.cpp

Files Outline <| builtins.cpp

```

1161 static Result<void> do_installkey(const BuiltinArguments& args) {
1162     if (!is_file_crypto()) return {};
1163
1164     auto unencrypted_dir = args[1] + fsencrypt_unencrypted_folder;
1165     if (!make_dir(unencrypted_dir, 0700) && errno != EEXIST) {
1166         return ErrnoError() << "Failed to create " << unencrypted_dir;
1167     }
1168     return ExecVdcRebootOnFailure("enablefilecrypto");
1169 }

```

最终，这个流程会进入 vold 中的 fbeEnable() 函数。

```
platform/superproject/main > main > system/vold/vdc.cpp
Files Outline vdc.cpp
vold
bench
binder
159 auto vold = android::interface_cast<android::os::IVold>(binder);
160
161 if (args[0] == "cryptfs" && args[1] == "enablefilecrypto") {
162     checkStatus(args, vold->fbeEnable());
163 } else if (args[0] == "cryptfs" && args[1] == "init_user0") {
164     checkStatus(args, vold->initUser0());
```

一路执行后，流程会进入 vold 中的相关 API。

xref: /system/vold/VoldNativeService.cpp

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download Search

```
557         "mountId: " + std::to_string(mountId) + " fileId: " + std::to_string(fileId) +
558         " flags: " + std::to_string(flags));
559     }
560
561     *_aidl_return = android::base::unique_fd(fd);
562     return Ok();
563 }
564
565 binder::Status VoldNativeService::fbeEnable() {
566     ENFORCE_SYSTEM_OR_ROOT;
567     ACQUIRE_CRYPT_LOCK;
568
569     write_file_bootprof("vold:fbeEnable:START");
570     bool rtn = fscrypt_initialize_systemwide_keys();
571     write_file_bootprof(StringPrintf("vold:fbeEnable:END, rtn=%d", rtn).c_str());
572     return translateBool(rtn);
573 }
574
```

xref: /system/vold/FsCrypt.cpp

Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download Search ☐ current

```
480
481 bool fscrypt_initialize_systemwide_keys() {
482     bool rtn = false;
483     LOG(INFO) << "fscrypt_initialize_systemwide_keys";
484
485     if (!init_data_file_encryption_options()) return false;
486
487     KeyBuffer device_key;
488     android::vold::write_file_bootprof("vold:retrieveOrGenerateKey:START");
489     rtn = retrieveOrGenerateKey(device_key_path, device_key_temp, kEmptyAuthentication,
490                               makeGen(s_data_options), &device_key);
491     android::vold::write_file_bootprof(StringPrintf("vold:retrieveOrGenerateKey:END, rtn=%d", rtn).c_str());
492     if (!rtn)
493         return false;
494
495     // This initializes s_device_policy, which is a global variable so that
496     // fscrypt_init_user0() can access it later.
497     android::vold::write_file_bootprof("vold:install_storage_key(device_key):START");
498     rtn = install_storage_key(DATA_MNT_POINT, s_data_options, device_key, &s_device_policy);
499     android::vold::write_file_bootprof(StringPrintf("vold:install_storage_key:END, rtn=%d", rtn).c_str());
500     if (!rtn)
501         return false;
502 }
```

2 检查 FBE 相关的设定

2.1 检查配置文件

与 FBE 相关的配置文件有 ProjectConfig.mk 和 fstab，只有配置正确，FBE 才会正常开启。

2.1.1 ProjectConfig.mk

查看 /device/mediateksample/yourproject/ProjectConfig.mk，如下 3 个配置需要是 no/no/yes。

- MTK_ENCRYPTION_DEFAULT_OFF = no
- MTK_ENCRYPTION_FDE_TO_FBE = no
- MTK_ENCRYPTION_TYPE_FILE = yes

2.1.2 fstab

查看代码中 vendor/mediatek/proprietary/hardware/fstab/mtxxxx，或者在平台上敲命令

cat /vendor/etc/fstab.mtXXXX (XXXX 为平台代号)。

```
/dev/block/by-name/userdata /data f2fs
noatime,nosuid,nodev,discard,noflush_merge,fsync_mode=nobarrier,reserve_root=134217,resgid=1
065,inlinecrypt
wait,check,formattable,quota,latemount,resize,reservedsize=128m,checkpoint=fs,fileencryption
=aes-256-xts:aes-256-
cts:v2+inlinecrypt_optimized,keydirectory=/metadata/vold/metadata_encryption,fsverity
```

需要确认 data 分区有如上红色部分的配置参数。

2.2 验证 FBE 是否有开启

验证平台上的 FBE 是否有开启，可以通过查看 property，以及查看数据是否有加密。

2.2.1 查看 property

输入如下命令查看是否有这些 property。

```
getprop | grep crypt
```

如下情况表示 FBE 已开启。

```
[ro.crypto.metadata.enabled]: [true]
```

```
[ro.crypto.state]: [encrypted]
[ro.crypto.type]: [file]
[ro.crypto.uses_fs_ioc_add_encryption_key]: [true]
[ro.crypto.volume.filenamees_mode]: [aes-256-cts]
```

如下情况表示 FBE 未开启。

```
[ro.crypto.state]: [unsupported]
[ro.crypto.volume.filenamees_mode]: [aes-256-cts]
```

2.2.2 查看 CE 文件是否为密文

1. 查看当前是哪个 user。

```
am get-current-user
10
```

2. 设置锁屏密码之后重启。
3. 重启进到开机密码的界面，不要输入密码。
4. 通过 shell 命令访问该 user 的 CE 数据。

```
ls /data/user/10/ (此处以 user 10 示例)
```

5. 如果 FBE 正常开启，则在未解锁的状态下，看到的都是密文，说明已加密，如下图：

```
auto8678p1_64_bsp_vm:Y # ls /data/user/10
-rcy0AAAAADxnL-7cNwq3SDT-PKZF9T1of6BvczqYyLgyHJSHx0EV-B-tqdGdESn-0HDTNmwwqk
0okLqwAAAACZ3W18g4oFYrvGntKq_iR-_4tf4yn8G19vm0D9qkB1Hg
1bccygAAAABW2ybX_VN1j1zykYnZY7ex4PaP8eEwTVf2Cm0bIv3IA
1dS5gQAAAACBc8a2LS1mgjNsGAzfpH1t_dYBQd1LTF5byloI1Icdmg
1otG4gAAAACEyV_VKw10b9Zn49AYzULwVn30VgAkZPYWAjDfjlANA
28o2AgAAAABnQn4V3ofWp16Y_cX4en461kR35yZ0QBPD1YScwxTnNa
2A1jBgAAAAC-g7uQRRVET2VAq4uuF1CU
2g-lsgAAAAA1NCZ4V3rvw5ctr1cbRLj9CcmSVT9zE7qk0LStXNUImA
3Q6qcQAAAAAS4-mL-fxZR5aBmk1V_ka
3SuyrQAAAAAxm1bzPNmEv6uuTSMViEztZr8Nq45HnwhP3U5sWMPrG
3X_FNWAAAAAwUFGH5W2U2cN5YLLIHAimk9RV0qor6IHKwc16BzdVw
3nn9-wAAAF0f-B-Ev0x7gfcD1eFCap_i5ifUa1LBiS0opjEXpWfEg
```

3 FBE 相关的问题

3.1 如何确认是做加密还是解密

在开机 log 中搜索关键字 `encryptFstab` 和 `mountFstab`。

- `encryptFstab` 加密，刷机或恢复出厂设置之后，首次开机时进行
- `mountFstab` 解密，非首次开机都会进入解密流程

3.2 如何关闭 FBE 功能

修改 `/device/mediateksample/yourproject/ProjectConfig.mk`，如下 3 个配置都改为 `no`。

`MTK_ENCRYPTION_DEFAULT_OFF = no`

`MTK_ENCRYPTION_FDE_TO_FBE = no`

`MTK_ENCRYPTION_TYPE_FILE = no`

需要注意的是，关闭 FBE 仅用于 debug，理清是否由 FBE 带来的问题。

3.3 USB 全量升级，恢复出厂设置，第二次开机启动后 FBE Key 异常

1. 在恢复出厂设置后，设备相当于第一次开机。在这个过程中，Android 系统不会立即进行解密，而是首先进行加密操作。加密完成后，才会进行解密操作。如果在恢复出厂设置后的第一次开机时，发现 `userdata` 无法挂载，首先需要考虑在恢复出厂设置的过程中，`userdata` 是否受到了损坏。
2. 在恢复出厂设置后的第二次开机时，系统会尝试使用在第一次开机时生成的 `metadata key` 来解密 `userdata` 分区。如果在挂载时出现 `invalid magic` 错误，系统会进入 `metadata` 解密流程。

如果解密失败或出现异常，可能的原因包括：

- `Metadata Key` 不存在或异常：如果 `metadata key` 在第一次开机时没有正确生成或存储，解密过程将无法进行。
 - `Keymaster` 出错：`Keymaster` 负责管理加密密钥，如果它出现故障或无法正常工作，解密过程可能会失败。
 - 文件系统损坏：如果在上次关机时 `userdata` 分区正在写入数据，而系统突然断电，可能导致文件系统损坏，从而影响挂载和解密过程。
3. 在 `metadata` 解密成功后，`userdata` 分区可以成功挂载。接下来，在 FBE 启用阶段，系统会检测到 FBE 已开启，并执行 `installkey` 操作。在这个过程中，系统会将 `FBE Class Key` 和 `Keymaster Key` 以 `KeyBlob` 的形式存储到 `userdata` 分区。当系统非首次启动时，会从 `userdata` 分区提取 `FBE Class Key` 和 `Keymaster Key` 对应的

KeyBlob，并通过 TEEOS 的 Keymaster TA 进行解密。通常，FBE key 的问题较少出现，主要与 Keymaster 和 TEE 相关。如果出现问题，可能需要检查 Keymaster 的状态和 TEE 的配置，以确保密钥管理和解密过程的正常进行。

3.4 执行 metadata 加密时候，fs_mgr 卸载 data 分区失败

- **Metadata 加密过程：**通常在设备烧录后的第一次开机或恢复出厂设置后的第一次开机时执行。在 `mount_all --late` 阶段，系统会尝试挂载 data 分区并成功。如果检测到 FBE metadata 加密已开启，系统会先执行卸载流程。在卸载 data 分区时，如果失败，需要根据 `unmount failed` 的返回值进行调试。一般情况下，卸载失败是因为有进程正在使用 userdata 分区。然而，根据谷歌的原生流程，此时 Android 组件尚未启动，因此需要检查是否有定制化的进程在访问 data 分区。
- **处理挂载失败：**如果由于上述异常导致 userdata 分区挂载失败，谷歌的原生流程通常会在 `init_user0` 阶段失败，并进入恢复模式（recovery mode）。用户可以在恢复模式中执行恢复出厂设置（factory reset）来解决问题。这一机制帮助用户在遇到系统无法正常启动时，通过恢复出厂设置来重新配置系统。

3.5 执行 metadata 加密时候，异常断电，data 分区加密未完成

如果在加密过程中系统断电，导致 data 分区未完成加密，这类问题可能会导致 userdata 分区无法挂载。谷歌的原生流程通常会有一些机制来处理这种情况。

在这种情况下，系统可能会在 `init_user0` 阶段失败，并进入恢复模式（recovery mode）。用户可以在恢复模式中执行恢复出厂设置（factory reset）来解决问题。

谷歌可能会针对这类问题发布补丁（Google patch），以改善系统在异常情况下的处理能力。建议查看谷歌的更新日志或相关文档，以了解是否有针对这种情况的补丁或解决方案。

3.6 执行 metadata 加密过程中，metadata key 被破坏

整个 metadata 的加密过程是由谷歌的原生流程处理的。如果 metadata key 被破坏，通常需要检查用户操作或文件系统方面是否存在异常，例如异常断电或异常写入行为。由于 metadata 是一个读写（RW）分区，key 的破坏通常与文件系统的异常操作相关联。

如果由于这些异常导致 userdata 分区挂载失败，谷歌的原生流程通常会在 `init_user0` 阶段失败，并进入恢复模式（recovery mode）。在这种情况下，用户可以通过恢复模式执行恢复出厂设置（factory reset）来解决问题。

3.7 后续开机解密流程，如果异常如何恢复，怎样确认是加密问题还是解密问题，或是分区被破坏

分区损坏是导致挂载失败的常见问题之一。解密失败通常与 Keymaster 的错误有关。为了诊断这些问题，可以在开机日志中查找与 keystore 或 keymaster 相关的关键字，查看是否有相关的错误日志。

4 其他网上资料

因为 FBE 是 Android 原生的功能，网上有很多资料，以下是一些推荐的总结资料：

1. <https://source.android.com/docs/security/features/encryption/file-based?hl=zh-cn>
2. https://blog.csdn.net/cs_tech/article/details/127579028
3. <https://blog.csdn.net/feelabclihu/article/details/131016357>

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.